Analiza SOLID principa za Escape Room sistem “The Last Key”

1. Single Responsibility Principle (SRP)

SRP je zadovoljen jer svaka klasa ima jasno definisanu odgovornost:

- **Korisnik** upravlja podacima o korisnicima sistema (registracija, prijava, uloge).
- **EscapeRoom** opisuje sobe (naziv, težina, kapacitet, cijena).
- **Termin** se bavi rasporedom i dostupnošću termina.
- **Rezervacija** upravlja procesom rezervacije i povezivanjem korisnika i termina.
- **Obavijest** služi za slanje informacija korisnicima.
- **Podrska** omogućava komunikaciju korisnika sa sistemom.
- **Recenzija** predstavlja ocjene i komentare korisnika.
- **Placanje** upravlja finansijskim transakcijama.

Svaka klasa se može mijenjati nezavisno jer ima samo jednu odgovornost.

2. Open-Closed Principle (OCP)

Sistem omogućava proširenje bez izmjene postojećih klasa:

- Klasa **Korisnik** se može proširiti novim ulogama (npr. moderator) bez izmjene postojeće strukture.
- Nove vrijednosti za **Tezina** ili **TipObavijesti** mogu se dodati kroz enumeracije.
- Funkcionalnosti recenzija ili rezervacija mogu se proširiti dodavanjem novih atributa ili metoda.

Time je sistem otvoren za proširenje, a zatvoren za izmjene postojećeg koda.

3. Liskov Substitution Principle (LSP)

LSP je zadovoljen jer se svi specifični tipovi korisnika mogu koristiti kao osnovni tip:

- Različite uloge korisnika (npr. admin, korisnik) mogu se tretirati kao **Korisnik**.
- U svim klasama gdje se koristi `korisnikID` (Rezervacija, Recenzija, Podrska), može se koristiti bilo koji tip korisnika bez problema.

Time se osigurava zamjenjivost bez narušavanja funkcionalnosti sistema.

4. Interface Segregation Principle (ISP)

Iako interfejsi nisu eksplicitno definisani, dizajn omogućava njihovu podjelu:

- **IKorisnik** – registracija, prijava, pregled soba
- **IAdmin** – upravljanje sobama i terminima
- **IRezervacija** – rezervacija i otkazivanje
- **IPlacanje** – obrada plaćanja
- **IRecenzija** – dodavanje i pregled recenzija

Svaki korisnik koristi samo funkcionalnosti koje su mu potrebne.

5. Dependency Inversion Principle (DIP)

Sistem može koristiti apstrakcije umjesto konkretnih implementacija:

- **Placanje** može zavisiti od interfejsa (npr. IPaymentService), a ne od konkretne implementacije (kartica, PayPal).
- **Obavijest** može koristiti apstraktni servis za slanje (email, SMS).
- **Rezervacija** može koristiti apstraktni servis za validaciju termina.

Time se omogućava:

- lakše testiranje
- zamjena implementacija
- fleksibilnost sistema